



南開大學  
Nankai University

计算机学院  
数据安全实验报告

## 基于 Paillier 算法实现隐私信息获取

姓名：王一诺  
学号：2312331  
专业：计算机科学与技术

2026 年 4 月 27 日

## 目录

|          |                                  |          |
|----------|----------------------------------|----------|
| <b>1</b> | <b>实验要求</b>                      | <b>2</b> |
| <b>2</b> | <b>原理概述</b>                      | <b>2</b> |
| 2.1      | 基本概念 . . . . .                   | 2        |
| 2.2      | 半同态加密 . . . . .                  | 3        |
| 2.3      | 半同态 Paillier 方案 . . . . .        | 3        |
| <b>3</b> | <b>环境配置</b>                      | <b>4</b> |
| <b>4</b> | <b>实验过程</b>                      | <b>4</b> |
| 4.1      | 基于 phe 的简单示例验证 . . . . .         | 4        |
| 4.2      | 基于 Paillier 算法实现隐私信息获取 . . . . . | 6        |
| 4.3      | 扩展实验 . . . . .                   | 7        |
| <b>5</b> | <b>结论与实践心得</b>                   | <b>9</b> |

## 1 实验要求

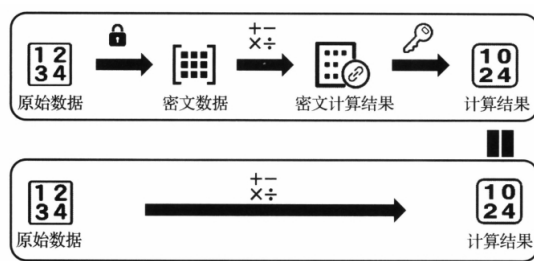
- 基于 Paillier 算法实现隐私信息获取: 从服务器给定的  $m$  个消息中获取其中一个, 不得向服务器泄露获取了哪一个消息, 同时客户端能完成获取消息的解密。
- 扩展实验: 有能力的同学, 可以在客户端保存对称密钥  $k$ , 在服务器端存储  $m$  个用对称密钥  $k$  加密的密文, 通过隐私信息获取方法得到指定密文后能解密得到对应的明文。

## 2 原理概述

### 2.1 基本概念

同态加密 (HE, homomorphic encryption) 是一种加密算法, 它可以通过对密文进行运算得到加密结果, 解密后与明文运算的结果一致, 如图所示。

同态加密主要基于公钥密码体制构建, 它允许将加密后的密文发给任意的第三方进行计算, 并且在计算前不需要解密, 可以在不需要密钥方参与的情况下, 在密文上直接进行计算。



同态加密方案由 KeyGen、Encrypt、Decrypt 和 Evaluate 4 个函数构成:

- $\text{KeyGen}(\lambda) \rightarrow (\text{pk}, \text{sk})$ : 密钥生成函数; 在给定加密参数  $\lambda$  后, 生成公钥/私钥对  $(\text{pk}, \text{sk})$ 。
- $\text{Encrypt}(\text{pt}, \text{pk}) \rightarrow \text{ct}$ : 加密函数; 使用给定公钥  $\text{pk}$  将目标明文数据  $\text{pt}$  加密为密文  $\text{ct}$ 。
- $\text{Decrypt}(\text{sk}, \text{ct}) \rightarrow \text{pt}$ : 解密函数; 使用给定密钥  $\text{sk}$  将目标密文数据  $\text{ct}$  解密为明文  $\text{pt}$ 。
- $\text{Evaluate}(\text{pk}, \Pi, \text{ct}_1, \text{ct}_2, \dots) \rightarrow (\text{ct}'_1, \text{ct}'_2, \dots)$ : 求值函数; 给定公钥  $\text{pk}$  与准备在密文上进行的运算函数  $\Pi$ , 求值函数将一系列的密文输入  $(\text{ct}_1, \text{ct}_2, \dots)$  转化为密文输出  $(\text{ct}'_1, \text{ct}'_2, \dots)$ 。

求值函数是同态加密方案不同于传统加密方案的部分。它的参数支持的运算函数种类决定了该同态加密方案支持的同态运算操作。在给定以上 4 个函数后, 同态加密方案应满足正确性、语义安全性和简短性。

根据同态加密算法所支持的同态操作种类和次数, 可以将现有同态加密方案分为以下几种类型:

- 半同态加密 (partial homomorphic encryption, 简称 PHE) 仅支持单一类型的密文域同态运算 (加或乘同态)。
- 类同态加密 (somewhat homomorphic encryption, 简称 SHE) 能够支持密文域有限次数的加法和乘法同态运算。
- 层级同态加密 (Leveled Homomorphic Encryption, 简称 LHE) 能同时支持多种同态操作 (加或乘同态), 并可以在安全参数中定义能够执行的操作次数上限。一般允许的操作次数越大, 该同态加密方案的密文空间开销及各类操作的时间复杂度就越大。

- 全同态加密 (fully homomorphic encryption, 简称 FHE) 能够实现任意次密文的加、乘同态运算。

## 2.2 半同态加密

- 乘法同态加密:是指存在有效算法  $F$ ,使得  $F(Enc(x), Enc(y)) = Enc(x \times y)$  或者  $Dec(F(Enc(x), Enc(y))) = x \times y$  成立, 并且不泄漏  $x$  和  $y$ 。

典型乘法同态加密算法是 RSA 算法和 ElGamal 算法。以 RSA 算法为例, 如果  $c_1 = m_1^e \bmod n$ ,  $c_2 = m_2^e \bmod n$ , 那么  $c_1 \times c_2 = m_1^e m_2^e \bmod n = (m_1 m_2)^e \bmod n = Enc(m_1 \times m_2)$ 。

- 加法同态加密:是指存在有效算法  $F$ ,使得  $F(Enc(x), Enc(y)) = Enc(x + y)$  或者  $Dec(F(Enc(x), Enc(y))) = x + y$  成立, 并且不泄漏  $x$  和  $y$ 。

典型加法同态加密算法是 Paillier 算法, 详见下节描述。

注意: 加法和乘法同态是相对明文而言所执行的操作, 而非密文上执行的运算形式。

## 2.3 半同态 Paillier 方案

Paillier 加密算法是 Paillier 等人 1999 年提出的一种基于判定  $n$  阶剩余类难题的典型密码学加密算法, 具有加法同态性, 是半同态加密方案。

### 密钥生成

- 随机选择两个质数  $p$  和  $q$ , 尽可能地保证  $p$  和  $q$  的长度接近或相等 (安全性高);
- 计算  $n=pq$  和  $\lambda = \text{lcm}(p-1, q-1)$ , 其中  $\text{lcm}$  表示最小公倍数;
- 随机选择  $g \in Z_{n^2}^*$ , 考虑计算性能优化, 通常会选择  $g=n+1$ ;
- 计算  $\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$ , 其中  $L(x) = \frac{x-1}{n}$ ;
- 公钥为  $(n, g)$ ;
- 私钥为  $(\lambda, \mu)$ 。

**加密算法** 对于任意明文消息  $m \in Z_n$ , 任意选择一个随机数  $r \in Z_n^*$ , 计算得到密文  $c$ :

$$c = E(m) = g^m r^n \bmod n^2$$

注意: 密文  $c$  要比明文  $m$  长度要长。

**解密算法** 对于密文  $c \in Z_{n^2}^*$ , 计算得到明文  $m$ :

$$m = D(c) = L(c^\lambda \bmod n^2) * \mu \bmod n$$

### 3 环境配置

为了进行实验，需要在 Windows 下安装 python 开发环境，并安装好 phe 库。安装完毕后，输入命令 python 和 from phe import paillier 来验证测试环境：

```
(datasafe) D:\数据安全\labcode\lab1>python
Python 3.12.7 | packaged by conda-forge | (main, Oct 4 2024, 15:47:54) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from phe import paillier
>>>
```

可见 python 环境和 phe 库都安装成功且能正常使用，接下来就可以进行实验。

## 4 实验过程

### 4.1 基于 phe 的简单示例验证

安装好 phe 库之后，使用给出的演示代码：

Listing 1: phe 示例代码

```
1 from phe import paillier # 开源库
2 import time # 做性能测试
3
4 ##### 设置参数
5 print("默认私钥大小: ", paillier.DEFAULT_KEYSIZE)
6 #生成公私钥
7 public_key, private_key = paillier.generate_paillier_keypair()
8 # 测试需要加密的数据
9 message_list = [3.1415926,100,-4.6e-12]
10
11 ##### 加密操作
12 time_start_enc = time.time()
13 encrypted_message_list = [public_key.encrypt(m) for m in message_list]
14 time_end_enc = time.time()
15 print("加密耗时s: ",time_end_enc-time_start_enc)
16 print("加密数据 (3.1415926) :",encrypted_message_list[0].ciphertext())
17
18 ##### 解密操作
19 time_start_dec = time.time()
20 decrypted_message_list = [private_key.decrypt(c) for c in encrypted_message_list]
21 time_end_dec = time.time()
22 print("解密耗时s: ",time_end_dec-time_start_dec)
23 print("原始数据 (3.1415926) :",decrypted_message_list[0])
24
25 ##### 测试加法和乘法同态
26 a,b,c = encrypted_message_list # a,b,c分别为对应密文
27 a_sum = a + 5 # 密文加明文，已经重载了+运算符
28 a_sub = a - 3 # 密文加明文的相反数，已经重载了-运算符
29 b_mul = b * 6 # 密文乘明文,数乘
```

```

30 c_div = c / -10.0 # 密文乘明文的倒数
31
32 print("a+5 密文:",a.ciphertext()) # 密文纯文本形式
33 print("a+5=",private_key.decrypt(a_sum))
34 print("a-3",private_key.decrypt(a_sub))
35 print("b*6=",private_key.decrypt(b_mul))
36 print("c/-10.0=",private_key.decrypt(c_div))
37
38 ##密文加密文
39 print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a+b))
40 #报错, 不支持a*b, 即两个密文直接相乘
41 #print((private_key.decrypt(a)+private_key.decrypt(b))==private_key.decrypt(a*b))
42

```

运行结果如下所示:

```

(datasafe) D:\数据安全\labcode\lab1>python phe_test.py
默认私钥大小: 3072
加密耗时s: 0.4940071105957031
加密数据 (3.1415926) : 4794097538697087283266199095008001054800547039895281325802273660658257362536378606156375191718209
246115709013979081918472161412520808717601616189509621046569636214093366176197495848183349994239650145171153313632588953
575734586719435058373581110843838601525499581829419277502673679021129586090725737657695786977886736916095321086855407616
652262839080379569864395441133393216820847940834167915344388757732306089238758252410763943244501577507717295858736173975
205162059647838195645901147861387634042263510613361246871048698035861312825529171720237900097808526533930726642922511519
164815318397916356131069858700564523707417053202522027856107898209932797996958657089935186604987992552786826580940804377
800066238691397291009078053630940161704068257693255787458388990222176576520141911264478574047484687788891369080254547557
699110663885032276710762379731831357870706702780721809722139907017289274982889823910883817783280850815411957761721601346
319682828328392379701722516108315384166678102324441666827835572774007762076544952127334495600772713498296296938194398438
465676013691327435503011569504151203308911581235150781218104200003018451838985419752874751624026158993096788680146544318
813876578299159463239886414826990738298837215756598311877395383762960246151183249627949158520648590956136521686961270900
446380694962916585835985529645815022670375840841072958528551432082992208560729662675884194574911100951292264419070300239
377731082079929327449206419034917052891921463859068540834235713088780554718326743028094780944695778550902148930838027287
400229340314501205619663668785391290663093800169118882292363861410549186414782189354415402693986173417204141790084451195
429964464666215650993190853498414363998029397613140964063579801386166080731954879501915829676963531487451867825209241192
712042924074550485524226472859484375705927095241600371480652250987274159
解密耗时s: 0.14407324790995459
原始数据 (3.1415926) : 3.1415926
a+5 密文: 47940975386970872832661990950080010548005470398952813258022736606582573625363786061563751917182092461157090139
790819184721614125208087176016161895096210465696362140933661761974958481833499942396501451711533136325889535757345867194
350583735811108438386015254995818294192775026736790211295860907257376576957869778867369160953210868554076166522628390803
795698643954411333932168208479408341679153443887577323060892387582524107639432445015775077172958587361739752051620596478
381956459011478613876340422635106133612468710486980358613128255291717202379000978085265339307266429225115191648153183979
818334999423965014517115331363258895357573458671943505837358111084383860152549958182941927750267367902112958609072573765
769578697788673691609532108685540761665226283908037956986439544113339321682084794083416791534438875773230608923875825241
076394324450157750771729585873617397520516205964783819564590114786138763404226351061336124687104869803586131282552917172
023790009780852653393072664292251151916481531839791635613106985870056452370741705320252202785610789820993279799695865708
993518660498799255278682658094080437780006623869139729100907805363094016170406825769325578745838899022217657652014191126
447857404748468778889136908025454755769911066388503227671076237973183135787070670278072180972213990701728927498288982391
088381778328085081541195776172160134631968282832839237970172251610831538416667810232444166682783557277400776207654495212
733449560077271349829629693819439843846567601369132743550301156950415120330891158123515078121810420000301845183898541975
28747516240261589930967886801465443181387657829915946323988641482699073829883721575659831187739538376296024615118324962
794915852064859095613652168696127090044638069496291658583598552964581502267037584084107295852855143208299220856072966267
58841945749111009512922644190703002393773108207992932744920641903491705289192146385906854083423571308878055471832674302
809478094469577855090214893083802728740022934031450120561966366878539129066309380016911888229236386141054918641478218935
441540269398617341720414179008445119542996446466621565099319085349841436399802939761314096406357980138616608073195487950
1915829676963531487451867825209241192712042924074550485524226472859484375705927095241600371480652250987274159
a+5= 8.1415926
a-3 0.14159260000000007
b*6= 600
c/-10.0= 4.6e-13
True

```

可见成功完成了加密与解密, 且最后输出 true, 运行成功。

## 4.2 基于 Paillier 算法实现隐私信息获取

回顾一下实验要求：服务器端拥有多个数值，要求客户端能基于 Paillier 实现从服务器读取一个指定的数值并正确解密，但服务器不知道所读取的是哪一个。

首先，要基于 Paillier 协议进行设计。

对 Paillier 的标量乘的性质进行扩展，可知：数值“0”的密文与任意数值的标量乘也是 0，数值“1”的密文与任意数值的标量乘将是数值本身。

基于这个特性，我们可以如下巧妙的设计（由实验指导所提供）：

**服务器端：** 产生数据列表  $data\_list = m_1, m_2, \dots, m_n$

**客户端：**

- 设置要选择的数据位置为 pos。
- 生成选择向量  $select\_list = \{0, \dots, 1, \dots, 0\}$ ，其中，仅有 pos 的位置为 1。
- 生成密文向量  $enc\_list = \{E(0), \dots, E(1), \dots, E(0)\}$ 。
- 发送密文向量  $enc\_list$  给服务器。

**服务器端：**

- 将数据与对应的向量相乘后累加得到密文  $c = m_1 * enc\_list[1] + \dots + m_n * enc\_list[n]$ 。
- 返回密文 c 给客户端。

**客户端：** 解密密文 c 得到想要的结果。

使用提供的代码，如下：

Listing 2: 基于 Paillier 的隐私信息获取

```

1  from phe import paillier # 开源库
2  import random # 选择随机数
3
4  ##### 设置参数
5  # 服务器端保存的数值
6  message_list = [100,200,300,400,500,600,700,800,900,1000]
7  length = len(message_list)
8  # 客户端生成公私钥
9  public_key, private_key = paillier.generate_paillier_keypair()
10 # 客户端随机选择一个要读的位置
11 pos = random.randint(0,length-1)
12 print("要读起的数值位置为: ",pos)
13
14 ##### 客户端生成密文选择向量
15 select_list=[]
16 enc_list=[]
17 for i in range(length):
18     select_list.append( i == pos )

```

```

19 enc_list.append( public_key.encrypt(select_list[i]) )
20
21 # for element in select_list:
22 #     print(element)
23 # for element in enc_list:
24 #     print(private_key.decrypt(element))
25
26 ##### 服务器端进行运算
27 c=0
28 for i in range(length):
29     c = c + message_list[i] * enc_list[i]
30 print("产生密文: ",c.ciphertext())
31
32 ##### 客户端进行解密
33 m=private_key.decrypt(c)
34 print("得到数值: ",m)
35

```

运行结果如下所示:

```

(datasafe) D:\数据安全\labcode\lab1>python Paillier_get_privateinfo.py
要读起的数值位置为: 2
产生密文: 822029728479927684279338538640640905721932131223416299833911775927940967591024722673382661222736000714921423509
96225829672012542827081886677952409390659007414563207871409858265473588056967043981590930159140608901670493838448889260688
16187894488149623893883078820931146334852004795154579152573135865687302637670097792577206957024210456217423041507235227156
99921592994832149449055357714798796349770519023558627003737284752042749789340885330627103710885278881916291887484355335210
54546819653050096378956297610487542327791060803584581720023172851321224723366630067763994781608665936486571526071844696797
49333622582318923710604992808761314945228170780026812351190870852718471108999413201527550305694096858667655707883017033696
04436987258484812346818296666909796871694470549722400469600792791466094507585534192655287974482611599757485223379418072862
61392059659545678510342544506632070568897633587086666437966256908316962419697416536099956122360299485001054019526651472529
6601838045323524960227990675109984295180435249833032735543694202956205240222729019052468502628972544141023254826655589684
14148770492847419865957468299345941776846699042085471995710425881312594055329035283872243260112116642498316670530515485318
33298991183165278345891111865997456109042117940191118699293513560702976066645128069192092083466562126923402883544330900807
81473757768718794529325432316710057915068838319798756010566102354826967707626404727929502438514466161289118250866512543439
91937889313224481738595444819703233872505957960571888463498861897556529148280500163401609309451853453922944899237251669137
77353624667347246251088834055419815477850033480232893459265103144516413073268934988298032248956797984178328027979076393303
94403546448112907424110747590149398443694284093423377061795662363176087016540785891245927194066012700425934858725707698375
914250094409619745995053210230
得到数值: 300

```

可见成功拿到了正确的隐秘信息，实验成功。

### 4.3 扩展实验

回顾实验要求：有能力的同学，可以在客户端保存对称密钥 k，在服务器端存储 m 个用对称密钥 k 加密的密文，通过隐私信息获取方法得到指定密文后能解密得到对应的明文。

对称加密有很多种方法，这里我使用 DES 对称加密来完成该实验：

Listing 3: DES 对称加密

```

1 from phe import paillier # 开源库
2 import random # 选择随机数
3 import pyDes # DES加密
4
5 ##### 设置参数
6 # 服务器端保存的数值

```

```
7 plain_message_list = [100,200,300,400,500,600,700,800,900,1000]
8 # 客户端保存对称密钥k
9 k = "12345678"
10 des = pyDes.des(k, padmode=pyDes.PAD_PKCS5)
11 # 服务器端存储用对称密钥k加密后的密文，转为整数后参与Paillier同态运算
12 message_list = [int.from_bytes(des.encrypt(str(message).encode()), byteorder="big") for
13 message in plain_message_list]
14 length = len(message_list)
15 # 客户端生成公私钥
16 public_key, private_key = paillier.generate_paillier_keypair()
17 # 客户端随机选择一个要读的位置
18 pos = random.randint(0,length-1)
19 print("要读起的数值位置为: ",pos)
20 ##### 客户端生成密文选择向量
21 select_list=[]
22 enc_list=[]
23 for i in range(length):
24     select_list.append( i == pos )
25     enc_list.append( public_key.encrypt(select_list[i]) )
26
27 # for element in select_list:
28 #     print(element)
29 # for element in enc_list:
30 #     print(private_key.decrypt(element))
31
32 ##### 服务器端进行运算
33 c=0
34 for i in range(length):
35     c = c + message_list[i] * enc_list[i]
36     print("产生密文: ",c.ciphertext())
37
38 ##### 客户端进行解密
39 m=private_key.decrypt(c)
40 message_bytes = m.to_bytes(max(8, ((m.bit_length() + 63) // 64) * 8), byteorder="big")
41 m = int(des.decrypt(message_bytes).decode())
42 print("得到数值: ",m)
43
```

运行结果如下所示:

```
(datasafe) D:\数据安全\labcode\lab1>python PaillierDES_get_privateinfo.py
要读取的数值位置: 4
产生密文: 706822266684438968599768551836055330558831698383415807584327864709947540343427496764329956073761691165343
034738431756148173165158958380381102773231526349617209856524216809340312140388502095137992864911754969079019489711950
704071177227108092526643907289125172533480451778992129349225864539025440162945060434960758316756680840765039927623641
815810795409507471528979182463809346566237313304102934139258210271758729606972606878653678237510793506759551396562635
881625272683719852856126249116541916218643115638816952510076603678632590073527701498165260323055158466007097190457739
282818193004899906826647719307196572623500728119631695792360862390311611633302086538118235070184902742722661586449449
238745348816813505166609247118471775913546185409138510416374543337430211349770056136189032817540400558516298987902985
227674022552147969430880593403937031693502891031616600505238784287699159706067555353824225715858165129588635294940090
414089690738154437202151000168464003760063121053987779094474006189412527491369331808469737871155040107749809628900792
544726764850143825144771797570691701088576362256654587607652978231438279714800323715761947610864093356325227807599979
87685328247192316018034391871415769549357210828674093227640533603368976283751863999569569304215649988615815094153033
819226740810862641209717272217871721649915793979059227241029237647970187308312979424261370608701998447153431155336316
714380908962323136715681382641852118426237144792057954803268403899650853685896154226838540400026526299911614859775907
721201088016226207728431035487423330953361370445108700945339532054310637971156222017338197730622539655817357937568454
293577183116860811861333832283588461217421516551774766526601624532010522719067170628795617538659959038864674966378772
40143920602810319105429608524432003415316998635255970071191162729358401108639892640817839717680146017739
得到数值: 500
```

## 5 结论与实践心得

本次实验完成了基于 Paillier 半同态加密算法的隐私信息获取功能。客户端通过生成 Paillier 公私钥，并将选择向量加密后发送给服务器，使服务器能够在不知道客户端具体选择位置的情况下，对给定的多个消息进行同态计算。服务器返回计算结果后，客户端利用私钥解密即可得到目标消息，从而实现了“客户端获取指定消息，但服务器无法得知其访问的是哪一项”的实验目标。

在扩展实验中，进一步结合 DES 对称加密方法，对服务器端存储的数据进行了保护。客户端保存对称密钥  $k$ ，服务器端只保存由 DES 加密后的密文数据。客户端先通过 Paillier 隐私信息获取方法得到指定位置的 DES 密文，再使用本地保存的 DES 密钥进行解密，最终恢复出对应明文。这样不仅保护了客户端的访问隐私，也避免服务器直接保存明文数据，提高了整体数据存储的安全性。

通过本次实验，我加深了对 Paillier 加法同态性质的理解，认识到同态加密可以在不解密数据的情况下完成特定计算，非常适合隐私保护场景。同时也体会到，在实际应用中可以将公钥同态加密与对称加密结合使用：同态加密负责保护查询过程中的隐私，对称加密负责提高数据存储和解密效率。实验过程中还需要注意数据类型转换问题，例如 DES 加密结果是字节串，而 Paillier 运算对象是数值，因此需要在字节和整数之间进行转换。总体来说，本次实验加深了我对同态加密的理解，也让我感受到了数据加密和隐私保护的重要与魅力。